

# AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Faculty of Science and Technology



## Project

|                   |                          |                     |                                   |
|-------------------|--------------------------|---------------------|-----------------------------------|
| Assignment Title: | Personal Expense Tracker |                     |                                   |
| Assignment No:    | 01                       | Date of Submission: | 25 July 2026                      |
| Course Title:     | PROGRAMMING IN PYTHON    |                     |                                   |
| Course Code:      | CSC4162                  | Section:            | C                                 |
| Semester:         | Summer                   | 2025-26             | Course Teacher: MD. TANZEEM RAHAT |

### Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case-Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case-Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgement is made.
3. No part of this Assignment/Case-Study has been written for me/us by any other person except where such collaboration has been authorized by the concerned teacher and is clearly acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared and archived for the purpose of detecting plagiarism.
6. I/we give permission for a copy of my/our marked work to be retained by the faculty for review and comparison, including review by external examiners.
7. I/we understand that Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to expulsion from the University. Plagiarized material can be drawn from, and presented in, written, graphic and visual form, including electronic data, and oral presentations. Plagiarism occurs when the origin of their material used is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or to copy my/our work.

\* Student(s) must complete all details except the faculty use part.

\*\* Please submit all assignments to your course teacher or the office of the concerned teacher.

|                 |    |
|-----------------|----|
| Group Name/No.: | 03 |
|-----------------|----|

| No | Name                       | ID         | Program   | Signature |
|----|----------------------------|------------|-----------|-----------|
| 1  | SOUMIK DAS DIPON           | 23-51709-2 | BSc [CSE] |           |
| 2  | MD. MAHABUBUR RAHAMAN SIAM | 23-51706-2 | BSc [CSE] |           |
| 3  | DIPU ROY                   | 23-50420-1 | BSc [CSE] |           |
| 4  | NAHIDUL HASSAN             | 22-49252-3 | BSc [CSE] |           |

### Faculty use only

| FACULTY COMMENTS | Marks Obtained |  |
|------------------|----------------|--|
|                  |                |  |

# 1. Problem Statement and Objectives

## Problem Statement

Many people find it hard to keep track of their daily expense and income manually. Without a proper system, they cannot see their spending pattern, or manage the budget, or make any financial report easily. Other software that already exists for this is either too complicated or too costly for a normal user.

## Objectives

- Make a simple desktop app for recording the income and expense of a person.
- Let user categorize their transaction and also manage different payment methods.
- Allow user to set a monthly budget for each category, and give warning if they go over it.
- Show the spending trend using pie chart, bar chart and line chart so it is easy to understand.
- User can make filtered report (daily, weekly, monthly, yearly, or by category) and export it as CSV.
- Data should stay saved using simple CSV file, no need for setting up any database.

# 2. Feature List and Target Users

## Feature List

- Add, Edit and Delete an expense with date, category, amount, payment method, description, notes and recurring flag.
- Add, Edit and Delete an income with date, source, amount and description.
- Dashboard that shows summary card (total income, total expense, balance, budget remaining) and also the transaction list.
- Can search and filter the transaction by keyword or by category.
- Budget management: user sets monthly limit for each category, gets warning if amount is over the budget.
- Analytics page:
  - Show statistics like total, average, median, highest, lowest, standard deviation and variance.
  - Pie chart for spending by category, bar chart for monthly comparison, line chart for the trend.
  - A table showing category percentage and count.
- Reports page: user can generate report by day, week, month, year or category, then export as CSV.
- Settings page:
  - Manage category, add new one or delete (default category cannot be deleted).
  - Manage budget for each category.
  - Import or Export the expenses as CSV file.
- Menu bar on top so user can go to any section quickly.
- Validation for the input, like date format, amount must be positive number, category must be valid, etc.

## Target Users

This app is for normal people who just want to keep track of their own money, without using some complicated software or paying for online service.

# 3. Explanation of Used Python Concepts from Week 1–5

## Week 1 (Basic Syntax, Variables, I/O)

- Variable is used for storing the input from user, like date, amount, description.
- `print()` and `input()` is used indirectly, through the GUI entry widget instead of console.

## Week 2 (Control Flow: if-else, for, while)

- `if-else` is used in the validator function like `validate_amount` and `validate_date` to check the condition.
- `for` loop is used to go through the list of expense in analytics part and search part.

- while loop is used for build the list of year, inside `build_year_options` function.

### Week 3 (Functions)

- Function is made modular, example `validate_date()`, `format_currency()`, `total_expense()`.
- Some function have default parameter also, example `get_recent_expenses(n=10)`.

### Week 4 (Lists, Tuples, Sets, Dictionaries)

- List: for expense list, category name list, month key list.
- Dictionary: for budget lookup, month total, category percentage.
- Tuple: used for report field and file column name (implicitly).
- Set: not really used much in this project, maybe it could be used to collect unique category, but here list is enough.

### Week 5 (File Handling, Exception Handling)

- File handling: `open()`, `csv.DictReader`, `csv.DictWriter` is used for read and write the CSV data.
- Exception handling: try-except is used when we convert amount into float, when opening a file, or checking if file already exist (`FileNotFoundError`).

## 4. Data Structure Explanation: list, tuple, set, and dictionary

### List

- When loading all the expense from CSV: `rows = storage.load_expenses()` this return a list of dictionary.
- Category names is collect into a list: `category_names = []`.
- Month keys is sort for the chart axis: `sorted_months = list(month_totals.keys())`.

### Tuple

- Constant for payment method: `PAYMENT_METHODS = ("Cash", "Card", "Bank", "Mobile Banking", "Other")`.
- Column name is define as tuple: `columns = ("date", "category", "amount", "payment", "description")`.
- Function `get_month_range()` also return a tuple like `(start_date, end_date)`.

### Set

Set is not used directly anywhere in this project. It could be used maybe to collect unique category faster, but instead this project just use list with a membership check.

### Dictionary

- Every CSV row is basically a dictionary, and the key is same as the column header.
- `month_totals: { "YYYY-MM": total }` this dictionary is used for chart data.
- `category_percentages: { "Food": 35.5, ... }` also same kind of dictionary.
- `budget_entries` is a dictionary inside settings page, it map each category name to its own Entry widget.
- `FILE_PATHS` is a constant dictionary that map each data type to its file path.
- `budget_status` is a dictionary that store used amount, limit, remaining and over-budget flag.

## 5. OOP Design: class name, attributes, and methods

| Class                       | Attributes                                                                   | Methods / Notes                          |
|-----------------------------|------------------------------------------------------------------------------|------------------------------------------|
| Expense (models/expense.py) | id, date, category, amount, payment_method, description, notes, is_recurring | to_dict(), @staticmethod from_dict(data) |

|                                                                                    |                                                               |                                                                                        |
|------------------------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Income (models/income.py)                                                          | id, date, source, amount, description                         | to_dict(), @staticmethod from_dict(data)                                               |
| Category (models/category.py)                                                      | id, name, is_default                                          | to_dict(), @staticmethod from_dict(data)                                               |
| ExpenseTrackerApp (app.py)<br>- this is main window, inherit tk.Tk                 | container, frames (dictionary of all page frame)              | __init__(), build_menu_bar(), build_frames(), show_frame(name), and menu event handler |
| DashboardFrame (gui/dashboard.py) - main dashboard, inherit tk.Frame               | app, summary card, table, search widget, budget label         | build_...(), refresh_summary(), on_add_expense(), on_edit_selected(), etc              |
| AddExpenseWindow / AddIncomeWindow - these are popup form, inherit tk.Toplevel     | form entry widget, error label, optional expense_id/income_id | build_form(), validate_form(), on_save(), populate_for_edit()                          |
| AnalyticsFrame, ReportsFrame, SettingsFrame - page frame, all inherit tk.Frame     | own UI part for each page                                     | Each one have own build/refresh/generate logic inside                                  |
| SummaryCard (gui/widgets.py) - small reusable card widget                          | title_label, value_label                                      | update_value(new_value)                                                                |
| ValidatedEntry - entry box that check value on focus out (not use much in project) | validate_function                                             | on_focus_out(event), is_valid()                                                        |
| CategoryDropdown - Combobox that fill category by itself                           | (inherit ttk.Combobox)                                        | refresh_categories()                                                                   |

## 6. File Handling and Exception Handling Explanation

### File Handling

All the data is kept persistent using CSV file, inside the data/ folder.

- services/storage.py file have these function:
  - ensure\_data\_files\_exist() - this make the file with header if it not already there.
  - load\_expenses(), save\_expenses() - for reading and writing the expense record.
  - load\_income(), save\_income() - for reading and writing the income record.
  - load\_categories(), save\_categories() - for handling category.
  - load\_budgets(), save\_budgets() - for handling the budget limit.
- Every load function is using csv.DictReader to get the row as dictionary, then it convert the type, example int(row["id"]) or float(row["amount"]).
- Every save function use csv.DictWriter together with a fixed list of fieldnames.
- The export and import feature in settings page and report page also use csv.DictWriter / csv.DictReader along with the file dialog box.

### Exception Handling

- Inside ensure\_data\_files\_exist(): try: open(...) except FileNotFoundError: create file.
- Inside validate\_amount(): try: float(amount) except (ValueError, TypeError): return False.

- Inside `validate_date():` `try: datetime.strptime(...) except ValueError: return False.`
- Inside `on_save_budgets():` `try: value_number = float(value_text) except ValueError: show error.`
- We did not use any generic except without giving exact exception type, which we think is a good habit for coding.

## 7. Library/Module Use: NumPy, Tkinter, Matplotlib

### Tkinter

- This is used for building whole GUI - main window, frame, button, label, entry, combobox, treeview and menu.
- `tk.Tk` is the root window, `tk.Frame` is used for each page, `tk.Toplevel` is used for the popup form.
- `ttk.Combobox` for the dropdown, `ttk.Treeview` for showing table, `ttk.Notebook` for the tab.

### NumPy

- This is used inside `services/analytics.py`:
  - `np.array(amounts)` turn the normal list into array.
  - Function used: `np.sum`, `np.mean`, `np.median`, `np.std`, `np.var`, `np.max`, `np.min`.
- This make the statistic calculation faster and more easy to write.

### Matplotlib

- It connect with tkinter using `FigureCanvasTkAgg` so the chart can show up inside the app window.
- Used inside `AnalyticsFrame`:
  - `Figure` and `Axes` object is used for draw the pie, bar and line chart.
  - It show the category percentage, monthly comparison, and also the spending trend.

### CSV module

This is a standard python library, used for reading and writing the CSV file.

### datetime module

- `datetime.date.today()`, `datetime.timedelta`, `strptime` is used for doing the date calculation.

### tkinter.messagebox and filedialog

These is used for showing success, error or info message box, and also for the file open/save dialog.

## 8. Screenshots and Sample Input/Output

- Dashboard page with some sample expense/income, summary card and the transaction table.

**Personal Expense Tracker**

Income (This Month) **\$4,000.00** | Expense (This Month) **\$1,250.00** | Balance (This Month) **\$2,750.00** | Budget Remaining (Budgeted Categories) **\$5,420.00**

Budget Remaining only those categories that have a budget set in Settings.  
This Month Income \$4,000.00 | Expense \$1,250.00  
Budgets tracked: Food, Transport, Salary (all within limit)

Search:  Category:

**Recent Transactions (all dates - not limited to this month)**

| Date       | Category  | Amount   | Payment | Description        |
|------------|-----------|----------|---------|--------------------|
| 2020-07-25 | Transport | \$100.00 | Cash    | Bus cost           |
| 2020-07-20 | Gaming    | \$300.00 | Cash    | game price         |
| 2020-07-18 | Food      | \$250.00 | Cash    | Groceries          |
| 2020-07-18 | Food      | \$400.00 | Cash    | Lunch with friends |

**Recent Income**

| Date       | Source    | Amount      | Description    |
|------------|-----------|-------------|----------------|
| 2020-6-25  | Other     | \$10,000.00 | Part time work |
| 2020-12-12 | Freelance | \$10,000.00 |                |
| 2020-07-12 | Other     | \$4,000.00  |                |

- Add Expense window, with the field filled in and also a validation error being shown.

Manage

View

Help

Personal Expense Tracker

Expense (This Month)

Balance (This Month)

Budget Remaining (Budgeted Categories)

Date (YYYY-MM-DD)

2024-12-12

Expense (This Month)

\$1,250.00

Balance (This Month)

\$2,750.00

Budget Remaining (Budgeted Categories)

\$5,420.00

Budget

This Mo

Category

Transport

Search

Amount

-10

Recent

2024-01

2024-01

2024-01

2024-01

Amount must be a positive number.

Payment Method

Card

Description

Note

Add Ex

☒ Recurring

Recent

2024-01-12

2024-01-12

2024-01-12

Source

Vendor

Freelance

Other

Add Income

Edit Income

Delete Income

Category

Transport

Amount

\$100.00

Payment

Cash

Description

Bus cost

Amount

\$500.00

Payment

Cash

Description

game price

Amount

\$250.00

Payment

Cash

Description

Groceries

Amount

\$400.00

Payment

Cash

Description

Lunch with friends

Source

Vendor

Freelance

Other

Amount

\$10,000.00

Amount

\$10,000.00

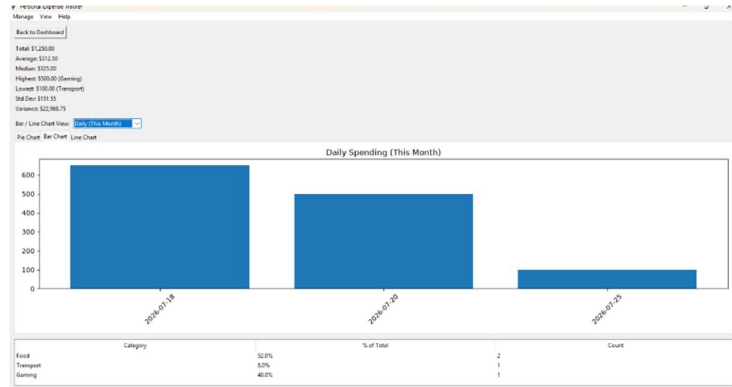
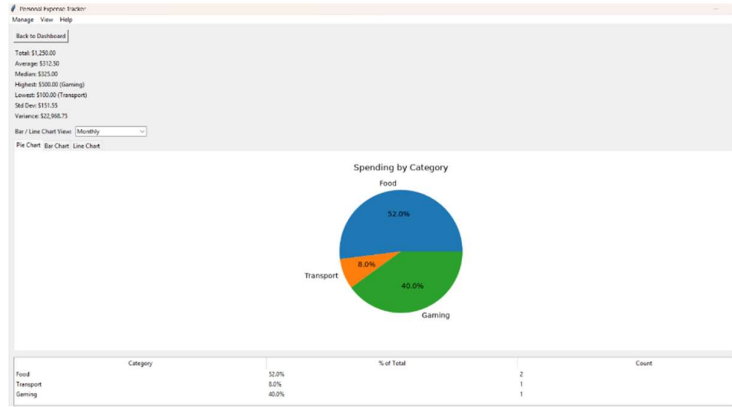
Amount

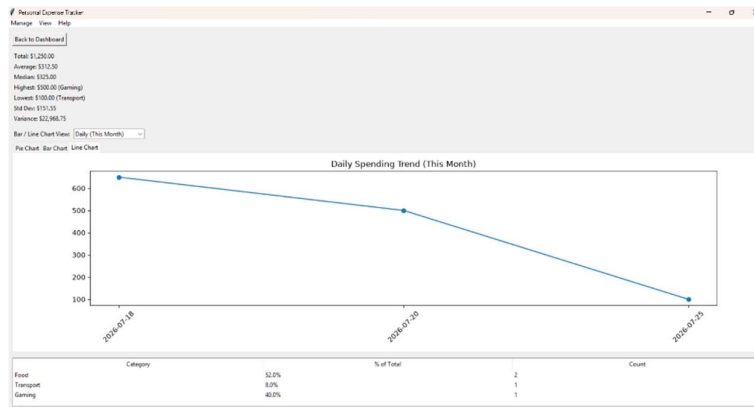
\$4,000.00

Part time work

Description

- Analytics page: pie chart, bar chart, line chart, the stats and category table.





- Reports page: after making a monthly report for "2026-07", showing the expense list and total.

Personal Expense Tracker

Manage View Help

[Back to Dashboard](#)

Report Type: Monthly

Months: 07 Year: 2026

| Date       | Category  | Amount   | Payment | Description        |
|------------|-----------|----------|---------|--------------------|
| 2026-07-18 | Food      | \$250.00 | Cash    | Groceries          |
| 2026-07-18 | Food      | \$400.00 | Cash    | Lunch with friends |
| 2026-07-20 | Transport | \$100.00 | Cash    | Bus cost           |
| 2026-07-20 | Gaming    | \$500.00 | Cash    | game price         |

Total: \$1,250.00 (4 transactions)

[Generate](#)

- Settings page: category list and the budget entries.

Personal Expense Tracker

Manage View Help

[Back to Dashboard](#)

**Categories**

Food (default)  
 Transport (default)  
 Shopping (default)  
 Entertainment (default)  
 Education (default)  
 Medical (default)  
 Bills (default)  
 Rent (default)  
 Salary (default)

[Add Category](#)  
[Delete Selected](#)

**Budgets**

Food:   
 Transport:   
 Shopping:   
 Entertainment:   
 Education:   
 Medical:   
 Bills:   
 Rent:   
 Salary:   
 Investment:   
 Others:   
 Gaming:   
 Savings:

[Save Budgets](#)

- A CSV file that already exported, opened inside a spreadsheet program.

Budget csv open in excel :

| category  | monthly_limit |
|-----------|---------------|
| Food      | 5000          |
| Transport | 1000          |
| biriyani  | 170           |

Category csv open in excel :

| id | name          | is_default |
|----|---------------|------------|
| 1  | Food          | TRUE       |
| 2  | Transport     | TRUE       |
| 3  | Shopping      | TRUE       |
| 4  | Entertainment | TRUE       |
| 5  | Education     | TRUE       |
| 6  | Medical       | TRUE       |
| 7  | Bills         | TRUE       |
| 8  | Rent          | TRUE       |
| 9  | Salary        | TRUE       |
| 10 | Investment    | TRUE       |
| 11 | Others        | TRUE       |
| 12 | Gaming        | FALSE      |
| 13 | biriyani      | FALSE      |

Expense csv open in excel :

| id    | category  | amount | payment_method | description           | notes | is_recurring |
|-------|-----------|--------|----------------|-----------------------|-------|--------------|
| ##### | Food      | 250    | Cash           | Groceries             |       | FALSE        |
| ##### | Food      | 400    | Cash           | Lunch with updated ar |       | TRUE         |
| ##### | Transport | 100    | Cash           | Bus cost              |       | TRUE         |
| ##### | Gaming    | 500    | Cash           | game price            |       | FALSE        |



Expense csv open in excel :

| A  | B  | C          | D         | E      | F              | G | H | I | J | K |
|----|----|------------|-----------|--------|----------------|---|---|---|---|---|
| 1  | id | date       | source    | amount | description    |   |   |   |   |   |
| 2  | 2  | 2026-07-25 | Other     | 10000  | Part time work |   |   |   |   |   |
| 3  | 3  | 2026-07-20 | Other     | 4000   |                |   |   |   |   |   |
| 4  | 4  | 2026-07-18 | Freelance | 10000  |                |   |   |   |   |   |
| 5  |    |            |           |        |                |   |   |   |   |   |
| 6  |    |            |           |        |                |   |   |   |   |   |
| 7  |    |            |           |        |                |   |   |   |   |   |
| 8  |    |            |           |        |                |   |   |   |   |   |
| 9  |    |            |           |        |                |   |   |   |   |   |
| 10 |    |            |           |        |                |   |   |   |   |   |
| 11 |    |            |           |        |                |   |   |   |   |   |
| 12 |    |            |           |        |                |   |   |   |   |   |
| 13 |    |            |           |        |                |   |   |   |   |   |
| 14 |    |            |           |        |                |   |   |   |   |   |
| 15 |    |            |           |        |                |   |   |   |   |   |
| 16 |    |            |           |        |                |   |   |   |   |   |
| 17 |    |            |           |        |                |   |   |   |   |   |

Sample Input/Output

Input: Expense (date: "2026-07-25", category: "Food", amount: 45.50, payment: "Card", description: "Lunch")

Income (This Month)  
\$4,000.00

Expense (This Month)  
\$1,250.00

Balance (This Month)  
\$2,750.00

Budget Remaining (Budgeted Categories)  
\$5,420.00

Add Expense

Date (YYYY-MM-DD)  
2026-07-25

Category  
Food

Amount  
45.50

Payment Method  
Card

Description  
Lunch

Notes

Recent Transactions

| Date       | Amount | Description        |
|------------|--------|--------------------|
| 2026-07-25 |        | Bus cost           |
| 2026-07-20 |        | game price         |
| 2026-07-18 |        | Groceries          |
| 2026-07-18 |        | Lunch with friends |

Recent Income

| Date       | Description    |
|------------|----------------|
| 2026-6-25  |                |
| 2026-12-12 | Part time work |
| 2026-07-12 |                |

Add Expense

Edit

Add Income

Edit Income

Delete Income

Output: Dashboard shows the updated total expense, a new row added in table, and budget card also get updated.

Income (This Month)  
\$4,000.00

Expense (This Month)  
\$1,295.50

Balance (This Month)  
\$2,704.50

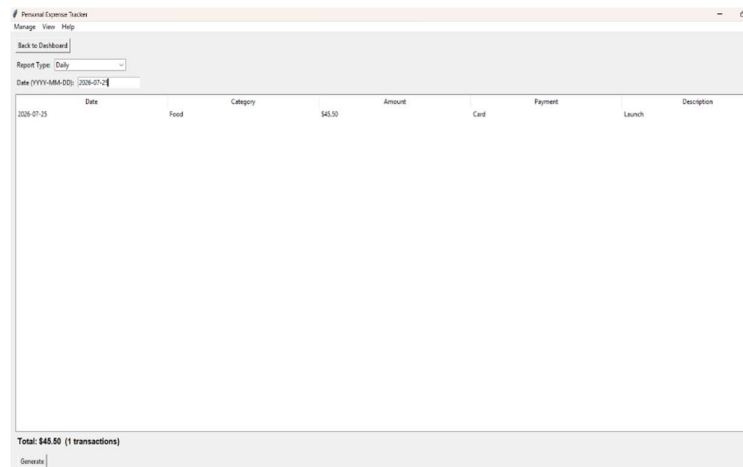
Budget Remaining (Budgeted Categories)  
\$5,374.50

Budget Remaining only totals categories that have a budget set in Settings.  
This Month: Income \$4,000.00 | Expense \$1,295.50  
Budgets tracked: Food, Transport, biryani (all within limit)  
Search: Category: Clear

Recent Transactions (all dates — not limited to this month)

| Date       | Category  | Amount   | Payment | Description        |
|------------|-----------|----------|---------|--------------------|
| 2026-07-25 | Transport | \$100.00 | Cash    | Bus cost           |
| 2026-07-25 | Food      | \$45.50  | Card    | Lunch              |
| 2026-07-20 | Gaming    | \$300.00 | Cash    | game price         |
| 2026-07-18 | Food      | \$250.00 | Cash    | Groceries          |
| 2026-07-18 | Food      | \$400.00 | Cash    | Lunch with friends |

Report generation: input the daily date "2026-07-25" → table shows all expense from that day, total \$45.50.



Personal Expense Tracker

Manage View Help

Back to Dashboard

Report Type: Daily

Date (YYYY-MM-DD): 2026-07-25

| Date       | Category | Amount  | Card | Payment | Launch | Description |
|------------|----------|---------|------|---------|--------|-------------|
| 2026-07-25 | Food     | \$45.50 |      |         |        |             |

Total: \$45.50 (1 transactions)

Generate

## 10. Limitations and Possible Future Improvements

### Limitations

- There is no login system, so anyone who open the app can see all the data.
- Data is only saved in local CSV file, which is not really suitable for multi-user or accessing remotely.
- Income tracking is quite limited, there no analytics specific for income source.
- Date have to be typed manually, so it easy to make mistake, since there no date picker.
- Budget only allow one limit per category, cannot set a different limit for different month.
- There no automation for recurring transaction, only a flag is there to mark it.
- The UI is not responsive, so it might look bad on different screen size.
- The charts don't have interactivity, like zoom or tooltip.

### Possible Future Improvements

- Add a proper login system, and maybe cloud sync using SQLite or some web API.
- Change from CSV into a lightweight database, for better handling of concurrency and query.
- Add a date picker widget so date entry is more easy.
- Make the recurring transaction automatic, so it auto add based on schedule.
- Improve the analytics part more, like predicting future spending, or comparing income against expense in a graph.
- Give an option in settings for currency symbol and date format.
- Add a backup and restore feature for the data.
- Maybe convert whole thing into a web application using Flask or Django, so more people can access it.